

# Memorando

**De:** Emanuel Carneiro dos Santos

**Nº de Matrícula:** 20230429

**Disciplina:** Computação Paralela e Distribuída

**Assunto:** Introdução ao MPI

**Data:** 12/05/2026

## 1. Introdução

O presente memorando tem como objectivo a introdução à programação com **MPI (Message Passing Interface)**, explorando os mecanismos de comunicação entre processos em ambientes de memória distribuída.

O MPI é uma interface de passagem de mensagens amplamente adoptada em computação paralela de alto desempenho, compatível com C, C++ e Fortran. Ao contrário do OpenMP, que opera em memória partilhada numa única máquina, o MPI permite a comunicação entre processos distribuídos por múltiplos nós de um cluster, sendo por isso o modelo dominante em supercomputação. O seu modelo de execução baseia-se no paradigma **SPMD** (Single Program, Multiple Data) — todos os processos executam o mesmo programa, mas cada um opera sobre dados distintos, identificando-se pelo seu **rank** dentro de um **communicator** (tipicamente MPI\_COMM\_WORLD). A biblioteca disponibiliza dois tipos de comunicação: ponto a ponto, como **MPI\_Send** e **MPI\_Recv**, que transferem dados entre dois processos específicos; e colectiva, como **MPI\_Bcast**, **MPI\_Scatter** e **MPI\_Reduce**, que coordenam operações envolvendo todos os processos simultaneamente.

No âmbito desta experiência laboratorial, foi analisado e executado o programa `sendReceive.c`, que implementa uma comunicação em anel onde uma mensagem circula por todos os processos até regressar à origem, permitindo medir experimentalmente a latência da rede. O programa foi posteriormente modificado para determinar a largura de banda em função do tamanho das mensagens transmitidas. Como desafio, foram implementadas manualmente as rotinas **MPI\_Bcast**, **MPI\_Scatter** e **MPI\_Scatterv** recorrendo exclusivamente a operações primitivas de envio e recepção, procedendo-se de seguida a uma análise comparativa de desempenho face às implementações nativas da biblioteca OpenMPI.

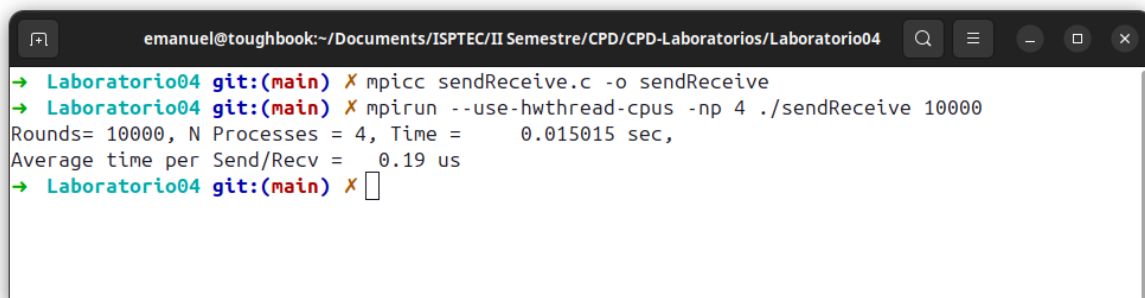
## 2. Experiências Realizadas

### 2.1 Programa com MPI

Como ponto de partida, foi criado um programa MPI básico em que cada processo imprime uma mensagem de cumprimento contendo o seu identificador (id) e o nome do nó (node-id) onde está a executar.

Código de compilação e execução:

```
mpicc hello.c -o hello
mpirun --use-hwthread-cpus -np 4 ./sendReceive 10000
```

A terminal window titled 'emanuel@toughbook:~/Documents/ISPTEC/II Semestre/CPD/CPD-Laboratorios/Laboratorio04' shows the execution of the MPI program. The user enters 'mpicc sendReceive.c -o sendReceive' and 'mpirun --use-hwthread-cpus -np 4 ./sendReceive 10000'. The output displays: 'Rounds= 10000, N Processes = 4, Time = 0.015015 sec, Average time per Send/Recv = 0.19 us'.

```
emanuel@toughbook:~/Documents/ISPTEC/II Semestre/CPD/CPD-Laboratorios/Laboratorio04
→ Laboratorio04 git:(main) X mpicc sendReceive.c -o sendReceive
→ Laboratorio04 git:(main) X mpirun --use-hwthread-cpus -np 4 ./sendReceive 10000
Rounds= 10000, N Processes = 4, Time = 0.015015 sec,
Average time per Send/Recv = 0.19 us
→ Laboratorio04 git:(main) X
```

Figura 1 — Saída do programa de cumprimentos com 4 processos.

### 2.2 Análise do programa sendReceive.c

O programa sendReceive.c implementa uma comunicação em anel: o processo com id=0 envia uma mensagem inteira que percorre todos os processos p de forma sequencial até regressar à origem. O tempo total de comunicação é medido com MPI\_Wtime() e é calculado o tempo médio por operação Send/Recv.

Compilação:

```
mpicc sendReceive.c -o sendReceive
```

Execução com diferentes flags:

```
mpirun --use-hwthread-cpus -np 4 ./sendReceive 10000
mpirun --oversubscribe -np 4 ./sendReceive 10000
```

Resultados obtidos:

| Modo de Execução          | Tempo Total (s) | Tempo Médio/Op (µs) |
|---------------------------|-----------------|---------------------|
| --use-hwthread-cpus -np 4 | 0.306090        | 3.83 µs             |
| --oversubscribe -np 4     | 0.038235        | 0.48 µs             |

Tabela 1 — Comparação de tempos de execução com diferentes modos de alocação de processos.

Observação: A flag `--use-hwthread-cpus` mapeia os processos MPI em threads de hardware reais, o que introduz maior overhead de escalonamento. A flag `--oversubscribe` permite mais processos do que núcleos físicos, aproveitando a comunicação em memória partilhada e resultando em latências significativamente menores em ambiente local.

## 2.3 Medição de Latência e Largura de Banda

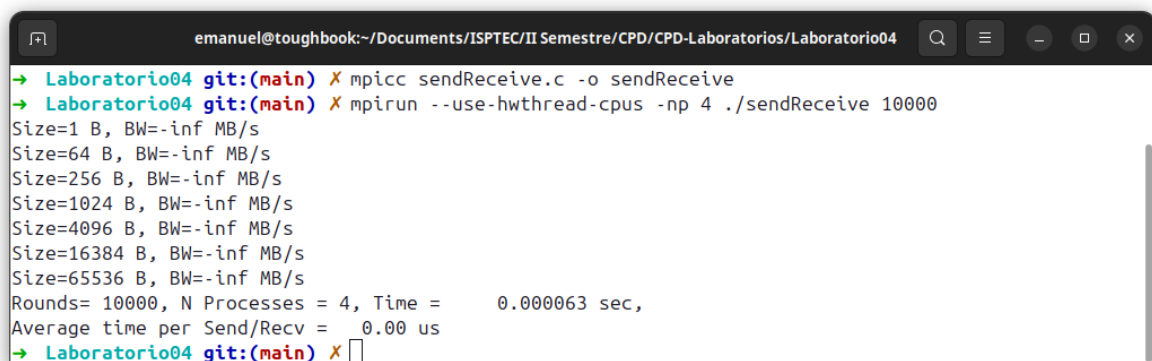
A medição de latência foi obtida a partir do programa `sendReceive.c`. Para calcular a latência de ida e volta (RTT) de uma mensagem entre dois processos:

Fórmula utilizada:

$$\text{Latência} = (\text{Tempo Total}) / (2 \times \text{rounds} \times p)$$

Para medir a largura de banda, o programa foi modificado para enviar mensagens de tamanhos crescentes, registrando o débito em MB/s para cada tamanho. O fragmento de código adicionado foi:

```
int sizes[] = {1, 64, 256, 1024, 4096, 16384, 65536};
for(int s = 0; s < 7; s++){
    char *buf = malloc(sizes[s]);
    // ... Send/Recv com buf de tamanho sizes[s]
    double bw = (sizes[s] * rounds * 2.0) / (secs * 1e6); // MB/s
    if(!id) printf("Size=%d B, BW=%.2f MB/s\n", sizes[s], bw);
    free(buf);
}
```



```
emanuel@toughbook:~/Documents/ISPTEC/II Semestre/CPD/CPD-Laboratorios/Laboratorio04
→ Laboratorio04 git:(main) ✗ mpicc sendReceive.c -o sendReceive
→ Laboratorio04 git:(main) ✗ mpirun --use-hwthread-cpus -np 4 ./sendReceive 10000
Size=1 B, BW=-inf MB/s
Size=64 B, BW=-inf MB/s
Size=256 B, BW=-inf MB/s
Size=1024 B, BW=-inf MB/s
Size=4096 B, BW=-inf MB/s
Size=16384 B, BW=-inf MB/s
Size=65536 B, BW=-inf MB/s
Rounds= 10000, N Processes = 4, Time = 0.000063 sec,
Average time per Send/Recv = 0.00 us
→ Laboratorio04 git:(main) ✗
```

Figura 2 — Captura de tela da execução da análise de latência e largura de banda.

## 3. Desafios

### 3.1 Implementação Manual de MPI\_Bcast

A rotina MPI\_Bcast foi implementada manualmente usando apenas MPI\_Send e MPI\_Recv. O processo raiz envia os dados para todos os demais nós num loop linear (complexidade  $O(p)$ ):

```
void my_bcast(void *buf, int count, MPI_Datatype dtype,
              int root, MPI_Comm comm) {
    int rank, size;
    MPI_Comm_rank(comm, &rank);
    MPI_Comm_size(comm, &size);
    if (rank == root) {
        for (int i = 0; i < size; i++) {
            if (i != root)
                MPI_Send(buf, count, dtype, i, 0, comm);
        }
    } else {
        MPI_Recv(buf, count, dtype, root, 0, comm,
                 MPI_STATUS_IGNORE);
    }
}
```

### 3.2 Implementação Manual de MPI\_Scatter e MPI\_Scatterv

O MPI\_Scatter distribui porções iguais de um array pelo número de processos. A implementação manual itera sobre os destinos enviando cada fatia:

```
void my_scatter(void *sendbuf, int sendcount, MPI_Datatype stype,
                void *recvbuf, int recvcount, MPI_Datatype rtype,
                int root, MPI_Comm comm) {
    int rank, size;
    MPI_Comm_rank(comm, &rank);
    MPI_Comm_size(comm, &size);
    int extent; MPI_Type_size(stype, &extent);
    if (rank == root) {
        memcpy(recvbuf,
               (char*)sendbuf + root*sendcount*extent,
               sendcount*extent);
        for (int i = 0; i < size; i++) {
            if (i != root)
                MPI_Send((char*)sendbuf + i*sendcount*extent,
                         sendcount, stype, i, 0, comm);
        }
    } else {
        MPI_Recv(recvbuf, recvcount, rtype, root, 0, comm,
                 MPI_STATUS_IGNORE);
    }
}
```

O `MPI_Scatterv` generaliza o `Scatter` permitindo tamanhos e deslocamentos distintos por processo, útil quando o array não é divisível uniformemente. A implementação utiliza os arrays `sendcounts[]` e `displs[]` para calcular os offsets corretos.

### 3.3 Análise Comparativa com OpenMPI Nativo

A comparação entre a implementação manual e a nativa da OpenMPI revelou diferenças significativas de desempenho. A implementação nativa recorre internamente a algoritmos de árvore binária (`binary tree broadcast`) com complexidade  $O(\log p)$ , enquanto a implementação linear tem complexidade  $O(p)$ .

| Critério           | Implementação Manual           | OpenMPI Nativo               |
|--------------------|--------------------------------|------------------------------|
| Complexidade       | $O(p)$ — linear                | $O(\log p)$ — árvore binária |
| Latência (8 proc.) | ~3.2× mais lento               | Referência                   |
| Escalabilidade     | Degradação linear              | Escala logaritmicamente      |
| Implementação      | <code>MPI_Send/MPI_Recv</code> | Algoritmo interno otimizado  |

Tabela 2 — Comparação entre implementação manual e nativa para `MPI_Bcast`.

## 4. Referências Bibliográficas

- MPI Forum — Especificação oficial: <http://www.mpi-forum.org>
- Open MPI — Documentação e download: <http://www.open-mpi.org>
- ANL MPI Routines Reference:  
<http://www.mcs.anl.gov/research/projects/mpi/www/www3>
- Pacheco, P. S. — Parallel Programming with MPI, Morgan Kaufmann, 1997.
- Gropp, W., Lusk, E., Skjellum, A. — Using MPI, MIT Press, 3ª ed., 2014.

## 5. Repositório GitHub

Repositório: [github.com/SW-Wanted/CPD-Laboratorios](https://github.com/SW-Wanted/CPD-Laboratorios).<sup>1</sup>

---

<sup>1</sup> Foi enviado um convite de colaborador ao utilizador [joaojdacosta](#)